

Frontend Architecture Review

An honest assessment of the current TaskFlowApp Angular 20 codebase — what is solid, what is redundant, and what is empty scaffolding.

Project	TaskFlowApp (Angular 20, standalone, CDD + per-feature Clean Architecture)
Scope	core / shared / layouts / features · auth slice · shared UI library · routing & state
Method	Static read of source vs. the agreed target architecture; no changes made
Reviewer	Claude (Fable 5)

Overall verdict — Foundation grade: A–

The backbone is genuinely well-built and shaped by real Clean Architecture discipline. The problems are not in the foundation — they are accumulated redundancy and a large amount of empty scaffolding that is starting to misrepresent what actually exists.

1 · What is genuinely good

These are the parts worth being proud of and protecting. They reflect the target architecture faithfully and, in several places, correct problems that existed earlier in the build.

- **Four-bucket separation.** `core / shared / layouts / features` is clean and the dependency direction points inward correctly — `shared` never reaches into `features`.
- **The auth vertical slice is textbook.** `auth.repository.ts` (HTTP only, CQRS-labelled) → `auth.facade.ts` (use-case orchestration) → `login-page.ts` (talks only to the facade). The page has zero knowledge of HTTP.
- **Session state is well-separated.** `auth.service.ts` owns “who is logged in” over a store + token service, so guards and interceptors have one source of truth. `rememberMe` → local vs. session storage is a nice touch.
- **Deliberate interceptor pipeline.** `auth` → `logging` → `loading` → `error`, and the error interceptor maps the `failureReason` envelope through a handler table rather than a switch.
- **Signals + facades instead of NgRx** is the right call at this scale. `ApiService` as a thin wrapper over the `APP_SETTINGS` token is correct.
- **Multi-portal routing done properly.** `Layout-as-route-parent` + `lazy loadChildren` + guards, with honest TODO comments instead of half-wired dead routes.

Why this matters

The expensive, hard-to-retrofit decisions — layering, dependency direction, state strategy, routing topology — are all correct. Everything below is cosmetic or additive by comparison.

2 · What is redundant or misplaced

This is the honest part. None of it threatens the core design, but each item adds drift that compounds if left unaddressed.

2.1 Duplicate chart components

`productivity-chart` and `project-health-chart` live in `organisms/` and hardcode their data. The newer `bar-chart-molecule` and `donut-chart-molecule` do the same job but are properly `input()`-driven. Worse: the old hardcoded charts are the ones wired to the dashboard, while the good input-driven ones are used only in Storybook. Pick one — the organism charts should be deleted or become thin wrappers over the molecules.

2.2 Inconsistent naming & a redundant suffix

Earlier molecules are `feature-card`, `pricing-card`, `master-card`. Newer ones are `bar-chart-molecule`, `deadline-molecule`, `notification-pop-up-molecule`. The folder is already `molecules/`, so the `-molecule` suffix adds nothing and now only half the components carry it.

2.3 Several “molecules” are really organisms — and not *shared*

The three modals (`task-detail-modal`, `create-project-modal`, `view-project-modal`) and `project-overview` compose many parts and hold form / toggle state — organism-level by atomic-design rules. They are also feature-specific (`dashboard / projects`), so by the project's own rule (“shared = truly reusable”) they arguably belong under `features/organization/.../presentation/components/`.

2.4 master-card is a god-component

One component with a @switch across four unrelated shapes (achievement / stat / task / project) behind a mega-config interface. That is four molecules wearing one trench coat — it fights the “one component, one job” principle the rest of the design respects.

2.5 The molecules are not composed into anything

dashboard.ts is `export class Dashboard {}` and imports only the two old organism charts — none of the ~10 new molecules. They live entirely in Storybook right now. Fine mid-build, but they are unproven until a real page wires them to real data.

3 · The empty-scaffolding problem

There are roughly **35 empty directories** (.gitkeep-only). Some form a reasonable roadmap, but two items actively misrepresent the codebase:

- **The “validations engine” is empty.**
shared/validations/{decorators, engine, metadata, models} are all just .gitkeep. Nothing is implemented and nothing imports it — it is aspirational, not real.
- **Whole feature slices are pre-carved but hollow.** features/admin, features/member, the entire projects slice, and dashboard's domain/data/application are empty — as are shared/{directives, pipes, models, utils} and the empty atom folders (avatars, badges, icons, spinners).

Recommendation

Pre-scaffolding every folder before there is code for it is a “.NET solution” habit that does not pay off the same way in a TypeScript / Angular repo. Empty folders rot and make the structure claim capabilities that are not there. Let folders appear when the first file needs them.

4 · Also worth knowing — build status

The app does not currently compile green

There is a pre-existing ReviewCardModel shape mismatch in public-landing-page.html (old name / designation / company vs. new reviewerName / ...). It has been failing every build this session and is unrelated to the molecule work.

5 · Scorecard

Area	Grade	Notes
Layering & dependencies	A	Inward direction respected; auth slice is textbook.
State management	A	Signals + facades; appropriate, no over-engineering.
Routing & portals	A–	Lazy, guarded, layout-as-parent; some routes still scaffolded.
HTTP / interceptors	A	Ordered pipeline; envelope-aware error mapping.

Shared UI library	B	Good coverage, but naming/leveling drift & god-component.
Component reuse	B-	Duplicate charts; molecules not yet composed.
Repo hygiene	C+	~35 empty dirs; "validations engine" is folders only.
Build health	C	Pre-existing ReviewCardModel error blocks a green build.

6 · Suggested priorities (no changes made yet)

#	Action	Effort	Payoff
1	Fix the ReviewCardModel mismatch so the app builds green.	Low	High
2	De-duplicate charts: retire the hardcoded organism charts in favour of the input-driven molecules.	Low	High
3	Compose the new molecules into the real dashboard page with live data.	Med	High
4	Move feature-specific modals/overview out of shared into the owning feature.	Med	Med
5	Standardise molecule naming (drop the -molecule suffix).	Low	Med
6	Split master-card into distinct molecules per variant.	Med	Med
7	Delete empty scaffolding (incl. the validations folders) until code needs it.	Low	Med

Bottom line

Foundation: A-. Auth slice, layering, interceptors, routing and state strategy are all right and worth being proud of. The drift is at the edges — duplicated charts, inconsistent molecule naming and leveling, a few feature-specific components sitting in shared, and a lot of empty structure (including a "validations engine" that is just folders). None of it is hard to clean up, and none of it threatens the core design.